GOVERNMENT OF INDIA • COPYRIGHT OFFICE

DEPARTMENT FOR PROMOTION OF INDUSTRY AND INTERNAL TRADE (DPIIT)

APPLICATION FOR COPYRIGHT REGISTRATION — FORM XIV

DOCUMENT 02

EXTRACT OF FRONTEND SOURCE CODE

Statutory Source Code Deposit under Rule 70(5) of the Copyright Rules, 2013 (First 10 Pages and Last 10 Pages of Voluminous Computer Software)

Statutory Category:	Computer Software / Literary Work (Section 2(o), Copyright Act, 1957)
Software Title:	ARB SoftTech AI Enterprise Voice Calling Agent Platform — Frontend Software
Application Package:	voxora-enterprise-ai-voice (Version 2.0.0 Production)
Languages & Tech:	TypeScript (v5.6.3), JavaScript (ES2022), React 18.3.1, Vite 6, Tailwind CSS
Target Authority:	Registrar of Copyrights, Copyright Office, New Delhi, India
Statutory Mandate:	Deposit under Rule 70(5), Copyright Rules, 2013 (First 10 & Last 10 Pages Format)
Date of Compilation:	October 2026

OFFICIAL NOTICE OF COPYRIGHT DEPOSIT

CONFIDENTIAL STATUTORY SUBMISSION: This official documentation package is prepared strictly for submission to the Registrar of Copyrights, Government of India, under Section 45 of the Copyright Act, 1957. All proprietary source code, user interface designs, and architecture representations herein reflect original works of authorship. Third-party open-source libraries are disclaimed under Section 52.

STATUTORY SOURCE CODE DEPOSIT SPECIFICATION

Pursuant to Rule 70(5) of the Copyright Rules, 2013 and the official procedural guidelines of the Indian Copyright Office (copyright.gov.in) governing registration of Computer Software works, this document contains the authentic, unredacted, printable source code of the frontend software application entitled **'ARB SoftTech AI Enterprise Voice Calling Agent Platform — Frontend Software'**.

RULE 70(5) VOLUMINOUS DEPOSIT RULE

STATUTORY COMPLIANCE MEMORANDUM ON VOLUMINOUS SOFTWARE:
Where a computer software work is voluminous and exceeds twenty (20) printed pages in its entirety, the Registrar of Copyrights permits the applicant to deposit the **First Ten (10) Pages** and the **Last Ten (10) Pages** of the source code. The complete frontend codebase of this application comprises **118 source files** containing **23,552 lines of code**, which would span over 400 printed pages. Accordingly, this document deposits the true, unaltered first 10 pages and last 10 pages. A complete exhaustive file inventory is recorded in the accompanying SOURCE_CODE_INVENTORY.xlsx.

STRUCTURE OF SOURCE CODE DEPOSIT

Deposit Section	Page Range in Document	Source Files Covered	Functional Subsystem
Part I: First 10 Pages	Pages 3 to 12 (10 Pages)	src/main.tsx src/app/App.tsx src/app/router.tsx src/app/providers/AuthProvider.tsx src/app/providers/BusinessProvider.tsx src/app/providers/ToastProvider.tsx	Application Bootstrap, Root Provider Hierarchy, Declarative Routing, Authentication State, Multi-Tenant Context.
Intermediate Omission	Page 13 (Notice)	105 Intermediate Feature & Component Files (Documented in Full Inventory)	Feature Modules, Live Call Monitors, Agent Prompt Studio, Leads Pipeline, Questionnaires, Audio Players.
Part II: Last 10 Pages	Pages 14 to 23 (10 Pages)	src/utils/quickDial.ts src/utils/excelExport.ts src/utils/agentPdfExport.ts src/utils/businessPdfExport.ts src/types/api.ts	Quick-Dial Helpers, Client-side Excel Reporting, PDF Dossier Exporters, Enterprise TypeScript Schema Contracts.
Verification Certificate	Page 24	All Repository Files	Cryptographic Checksums & Authenticity Attestation.

CERTIFICATE OF UNALTERED CODE: The Applicant affirms that the source code presented in this deposit is extracted directly from the actual active codebase. No code has been fabricated, simulated, redacted, or blocked out. All identifiers, comments, imports, and functional logic represent the genuine work.

PART I: SOURCE CODE DEPOSIT — FIRST TEN PAGES [PAGE 1 OF 10]

```
// ============================================================================
// FILE IDENTIFIER: src/main.tsx
// STATUTORY CLASS: Computer Programme / Literary Work (Section 2(o))
// ============================================================================
0001 | import React from 'react';
0002 | import ReactDOM from 'react-dom/client';
0003 | import App from './app/App';
0004 | import './styles/index.css';
0005 | import './styles/modern.css';
0006 |
0007 | // Purge any stale service workers or caches from previous localhost PWAs (e.g. NagarDarpan)
0008 | if (typeof window !== 'undefined') {
0009 |   if ('serviceWorker' in navigator) {
0010 |     navigator.serviceWorker.getRegistrations().then((registrations) => {
0011 |       for (const reg of registrations) {
0012 |         reg.unregister();
0013 |       }
0014 |     });
0015 |   }
0016 |   if ('caches' in window) {
0017 |     caches.keys().then((keys) => {
0018 |       for (const key of keys) {
0019 |         caches.delete(key);
0020 |       }
0021 |     });
0022 |   }
0023 | }
0024 |
0025 | ReactDOM.createRoot(document.getElementById('root')!).render(
0026 |   <React.StrictMode>
0027 |     <App />
0028 |   </React.StrictMode>
0029 | );


// ============================================================================
// FILE IDENTIFIER: src/app/App.tsx
// STATUTORY CLASS: Computer Programme / Literary Work (Section 2(o))
// ============================================================================
0001 | import React from 'react';
0002 | import { RouterProvider } from 'react-router-dom';
0003 | import { router } from './router';
0004 | import { ThemeProvider } from './providers/ThemeProvider';
0005 | import { AuthProvider } from './providers/AuthProvider';
0006 | import { QueryProvider } from './providers/QueryProvider';
0007 | import { ToastProvider } from './providers/ToastProvider';
0008 | import { BusinessProvider } from './providers/BusinessProvider';
0009 |
0010 | export const App: React.FC = () => {
0011 |   return (
0012 |     <ThemeProvider>
0013 |       <QueryProvider>
0014 |         <AuthProvider>
0015 |           <BusinessProvider>
0016 |             <ToastProvider>
0017 |               <RouterProvider router={router} />
0018 |             </ToastProvider>
0019 |           </BusinessProvider>
0020 |         </AuthProvider>
0021 |       </QueryProvider>
0022 |     </ThemeProvider>
0023 |   );
0024 | };
0025 |
0026 | export default App;
0027 |
```

PART I: SOURCE CODE DEPOSIT — FIRST TEN PAGES [PAGE 2 OF 10]

```
// ============================================================================
// FILE IDENTIFIER: src/app/router.tsx
// STATUTORY CLASS: Computer Programme / Literary Work (Section 2(o))
// ============================================================================
0001 | import React from 'react';
0002 | import { createBrowserRouter, Navigate } from 'react-router-dom';
0003 | import { AppShell } from '../components/layout/AppShell';
0004 |
0005 | // Features & Pages
0006 | import { DashboardPage } from '../features/dashboard/DashboardPage';
0007 | import { BusinessesPage } from '../features/businesses/BusinessesPage';
0008 | import { BusinessDetailPage } from '../features/businesses/BusinessDetailPage';
0009 | import { CallsPage } from '../features/calls/CallsPage';
0010 | import { ActiveCallsPage } from '../features/calls/ActiveCallsPage';
0011 | import { QueuePage } from '../features/calls/QueuePage';
0012 | import { AgentsPage } from '../features/agents/AgentsPage';
0013 | import { AgentWorkspacePage } from '../features/agents/AgentWorkspacePage';
0014 | import { QuestionnairesPage } from '../features/questionnaires/QuestionnairesPage';
0015 | import { QuestionnaireEditorPage } from '../features/questionnaires/QuestionnaireEditorPage';
0016 | import { ContactsPage } from '../features/contacts/ContactsPage';
0017 | import { LeadsPage } from '../features/leads/LeadsPage';
0018 | import { FollowupsPage } from '../features/followups/FollowupsPage';
0019 | import { ReportsPage } from '../features/reports/ReportsPage';
0020 | import { InvoicesPage } from '../features/invoices/InvoicesPage';
0021 | import { ModelConfigPage } from '../features/models/ModelConfigPage';
0022 | import { ModelLabPage } from '../features/model_lab/ModelLabPage';
0023 | import { HealthPage } from '../features/health/HealthPage';
0024 | import { SettingsPage } from '../features/settings/SettingsPage';
0025 | import { LookPage } from '../features/modern/LookPage';
0026 | import { ModernDashboard } from '../features/modern/ModernDashboard';
0027 | import { ModernLeads } from '../features/modern/ModernLeads';
0028 | import { ModernCalls } from '../features/modern/ModernCalls';
0029 | import { ModernAgents } from '../features/modern/ModernAgents';
0030 | import { ModernBusinesses } from '../features/modern/ModernBusinesses';
0031 | import { ModernContacts } from '../features/modern/ModernContacts';
0032 | import { ModernFollowups } from '../features/modern/ModernFollowups';
0033 | import { ModernReports } from '../features/modern/ModernReports';
0034 | import { ModernInvoices } from '../features/modern/ModernInvoices';
0035 | import { ModernQuestionnaires } from '../features/modern/ModernQuestionnaires';
0036 | import { ModernSettings } from '../features/modern/ModernSettings';
0037 | import { ModernActiveCalls } from '../features/modern/ModernActiveCalls';
0038 | import { ModernCalendar } from '../features/modern/ModernCalendar';
0039 |
0040 | export const router = createBrowserRouter([
0041 |   {
0042 |     path: '/',
0043 |     element: <AppShell />,
0044 |     children: [
0045 |       { index: true, element: <LookPage classic={<DashboardPage />} modern={<ModernDashboard...
0046 |       { path: 'businesses', element: <LookPage classic={<BusinessesPage />} modern={<ModernB...
0047 |       { path: 'businesses/:id', element: <BusinessDetailPage /> },
0048 |       { path: 'calls', element: <LookPage classic={<CallsPage />} modern={<ModernCalls />} /...
0049 |       { path: 'active-calls', element: <LookPage classic={<ActiveCallsPage />} modern={<Mode...
0050 |       { path: 'queue', element: <QueuePage /> },
0051 |       { path: 'model-lab', element: <ModelLabPage /> },
0052 |       { path: 'agents', element: <LookPage classic={<AgentsPage />} modern={<ModernAgents />...
0053 |       { path: 'agents/:id', element: <AgentWorkspacePage /> },
0054 |       { path: 'questionnaires', element: <LookPage classic={<QuestionnairesPage />} modern={...
0055 |       { path: 'questionnaires/:id', element: <QuestionnaireEditorPage /> },
0056 |       { path: 'campaigns', element: <Navigate to="/calls" replace /> },
0057 |       { path: 'contacts', element: <LookPage classic={<ContactsPage />} modern={<ModernConta...
0058 |       { path: 'leads', element: <LookPage classic={<LeadsPage />} modern={<ModernLeads />} /...
0059 |       { path: 'followups', element: <LookPage classic={<FollowupsPage />} modern={<ModernFol...
0060 |       { path: 'calendar', element: <ModernCalendar /> },
```

PART I: SOURCE CODE DEPOSIT — FIRST TEN PAGES [PAGE 3 OF 10]

```
0061 |       { path: 'reports', element: <LookPage classic={<ReportsPage />} modern={<ModernReports...
0062 |       { path: 'invoices', element: <LookPage classic={<InvoicesPage />} modern={<ModernInvoi...
0063 |       { path: 'settings/models', element: <ModelConfigPage /> },
0064 |       { path: 'settings/health', element: <HealthPage /> },
0065 |       { path: 'settings/telephony', element: <SettingsPage initialTab="telephony" /> },
0066 |       { path: 'settings/transcript', element: <SettingsPage initialTab="transcript" /> },
0067 |       { path: 'settings', element: <LookPage classic={<SettingsPage />} modern={<ModernSetti...
0068 |       { path: '*', element: <Navigate to="/" replace /> },
0069 |     ],
0070 |   },
0071 | ]);


// ============================================================================
// FILE IDENTIFIER: src/app/providers/AuthProvider.tsx
// STATUTORY CLASS: Computer Programme / Literary Work (Section 2(o))
// ============================================================================
0001 | import React, { createContext, useContext, useEffect, useState, useCallback } from 'react';
0002 | import apiClient from '../../services/api/client';
0003 |
0004 | interface AuthContextType {
0005 |   token: string | null;
0006 |   isAuthenticated: boolean;
0007 |   isAuthModalOpen: boolean;
0008 |   openAuthModal: () => void;
0009 |   closeAuthModal: () => void;
0010 |   setAuthToken: (token: string) => void;
0011 |   logout: () => void;
0012 |   verifyToken: (token: string) => Promise<boolean>;
0013 | }
0014 |
0015 | const AuthContext = createContext<AuthContextType | undefined>(undefined);
0016 |
0017 | export const AuthProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
0018 |   const [token, setTokenState] = useState<string | null>(() => localStorage.getItem('api_aut...
0019 |   const [isAuthModalOpen, setIsAuthModalOpen] = useState(false);
0020 |
0021 |   const setAuthToken = useCallback((newToken: string) => {
0022 |     localStorage.setItem('api_auth_token', newToken);
0023 |     setTokenState(newToken);
0024 |   }, []);
0025 |
0026 |   const logout = useCallback(() => {
0027 |     localStorage.removeItem('api_auth_token');
0028 |     setTokenState(null);
0029 |   }, []);
0030 |
0031 |   const verifyToken = async (testToken: string): Promise<boolean> => {
0032 |     try {
0033 |       await apiClient.get('/api/agents', {
0034 |         headers: {
0035 |           Authorization: `Bearer ${testToken}`,
0036 |           'X-API-Token': testToken,
0037 |         },
0038 |       });
0039 |       return true;
0040 |     } catch (err: any) {
0041 |       if (err.response?.status === 401) {
0042 |         return false;
0043 |       }
0044 |       // If backend is unreachable or returns other status, permit storing token
0045 |       return true;
0046 |     }
0047 |   };
0048 |
0049 |   useEffect(() => {
```

PART I: SOURCE CODE DEPOSIT — FIRST TEN PAGES [PAGE 4 OF 10]

```
0050 |     const handleAuthRequired = () => {
0051 |       setIsAuthModalOpen(true);
0052 |     };
0053 |
0054 |     window.addEventListener('voxora:auth_required', handleAuthRequired);
0055 |     return () => window.removeEventListener('voxora:auth_required', handleAuthRequired);
0056 |   }, []);
0057 |
0058 |   return (
0059 |     <AuthContext.Provider
0060 |       value={{
0061 |         token,
0062 |         isAuthenticated: !!token,
0063 |         isAuthModalOpen,
0064 |         openAuthModal: () => setIsAuthModalOpen(true),
0065 |         closeAuthModal: () => setIsAuthModalOpen(false),
0066 |         setAuthToken,
0067 |         logout,
0068 |         verifyToken,
0069 |       }}
0070 |     >
0071 |       {children}
0072 |     </AuthContext.Provider>
0073 |   );
0074 | };
0075 |
0076 | export const useAuth = (): AuthContextType => {
0077 |   const context = useContext(AuthContext);
0078 |   if (!context) throw new Error('useAuth must be used within AuthProvider');
0079 |   return context;
0080 | };


// ============================================================================
// FILE IDENTIFIER: src/app/providers/BusinessProvider.tsx
// STATUTORY CLASS: Computer Programme / Literary Work (Section 2(o))
// ============================================================================
0001 | import React, { createContext, useContext, useState, useEffect, useCallback } from 'react';
0002 | import { businessesApi } from '../../services/api/businesses';
0003 |
0004 | export interface Business {
0005 |   id: string;
0006 |   name: string;
0007 |   industry: string;
0008 |   description?: string;
0009 |   contactEmail?: string;
0010 |   contactPhone?: string;
0011 |   status: 'active' | 'inactive' | 'onboarding';
0012 |   assignedAgentIds: string[];
0013 |   createdAt: string;
0014 |   updatedAt?: string;
0015 | }
0016 |
0017 | interface BusinessContextType {
0018 |   businesses: Business[];
0019 |   selectedBusinessId: string; // 'all' or specific business id
0020 |   selectedBusiness: Business | null;
0021 |   isAllBusinesses: boolean;
0022 |   isLoading: boolean;
0023 |   refreshBusinesses: () => Promise<void>;
0024 |   selectBusiness: (id: string) => void;
0025 |   addBusiness: (business: Omit<Business, 'id' | 'createdAt'>) => Promise<Business>;
0026 |   updateBusiness: (id: string, updates: Partial<Business>) => Promise<void>;
0027 |   deleteBusiness: (id: string) => Promise<void>;
0028 |   assignAgentToBusiness: (businessId: string, agentId: string) => Promise<void>;
0029 |   unassignAgentFromBusiness: (businessId: string, agentId: string) => Promise<void>;
```

PART I: SOURCE CODE DEPOSIT — FIRST TEN PAGES [PAGE 5 OF 10]

```
0030 |   getBusinessForAgent: (agentId?: string | null) => Business | undefined;
0031 | }
0032 |
0033 | const STORAGE_KEY_SELECTED = 'voxora_active_business_id_v1';
0034 |
0035 | const BusinessContext = createContext<BusinessContextType | undefined>(undefined);
0036 |
0037 | export const BusinessProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
0038 |   const [businesses, setBusinesses] = useState<Business[]>([]);
0039 |   const [isLoading, setIsLoading] = useState<boolean>(false);
0040 |   const [selectedBusinessId, setSelectedBusinessId] = useState<string>(() => {
0041 |     try {
0042 |       const saved = localStorage.getItem(STORAGE_KEY_SELECTED);
0043 |       if (saved) return saved;
0044 |     } catch {
0045 |       // fallback
0046 |     }
0047 |     return 'all';
0048 |   });
0049 |
0050 |   const refreshBusinesses = useCallback(async () => {
0051 |     try {
0052 |       setIsLoading(true);
0053 |       const data = await businessesApi.list();
0054 |       if (Array.isArray(data)) {
0055 |         const mapped: Business[] = data.map((b) => ({
0056 |           id: b.business_id,
0057 |           name: b.name,
0058 |           industry: b.industry,
0059 |           contactEmail: b.email,
0060 |           contactPhone: b.phone,
0061 |           status: b.is_active ? 'active' : 'inactive',
0062 |           assignedAgentIds: b.assigned_agent_ids || [],
0063 |           createdAt: b.created_at,
0064 |         }));
0065 |         setBusinesses(mapped);
0066 |       }
0067 |     } catch (err) {
0068 |       console.warn('[BusinessProvider] Backend fetch notice:', err);
0069 |     } finally {
0070 |       setIsLoading(false);
0071 |     }
0072 |   }, []);
0073 |
0074 |   useEffect(() => {
0075 |     refreshBusinesses();
0076 |   }, [refreshBusinesses]);
0077 |
0078 |   // Persist active selection
0079 |   useEffect(() => {
0080 |     try {
0081 |       localStorage.setItem(STORAGE_KEY_SELECTED, selectedBusinessId);
0082 |     } catch {
0083 |       // ignore
0084 |     }
0085 |   }, [selectedBusinessId]);
0086 |
0087 |   const selectedBusiness =
0088 |     selectedBusinessId === 'all'
0089 |       ? null
0090 |       : businesses.find((b) => b.id === selectedBusinessId) || null;
0091 |
0092 |   const isAllBusinesses = selectedBusinessId === 'all';
0093 |
0094 |   const selectBusiness = (id: string) => {
0095 |     setSelectedBusinessId(id);
```

PART I: SOURCE CODE DEPOSIT — FIRST TEN PAGES [PAGE 6 OF 10]

```
0096 |   };
0097 |
0098 |   const addBusiness = async (data: Omit<Business, 'id' | 'createdAt'>): Promise<Business> => {
0099 |     try {
0100 |       const doc = await businessesApi.create({
0101 |         name: data.name,
0102 |         industry: data.industry,
0103 |         email: data.contactEmail,
0104 |         phone: data.contactPhone,
0105 |       });
0106 |       const newBiz: Business = {
0107 |         id: doc.business_id,
0108 |         name: doc.name,
0109 |         industry: doc.industry,
0110 |         contactEmail: doc.email,
0111 |         contactPhone: doc.phone,
0112 |         status: doc.is_active ? 'active' : 'inactive',
0113 |         assignedAgentIds: doc.assigned_agent_ids || [],
0114 |         createdAt: doc.created_at,
0115 |       };
0116 |       setBusinesses((prev) => [newBiz, ...prev]);
0117 |       return newBiz;
0118 |     } catch (err) {
0119 |       console.warn('[BusinessProvider] Backend create error, falling back locally:', err);
0120 |       const localId = `biz_${Date.now()}`;
0121 |       const newBiz: Business = {
0122 |         ...data,
0123 |         id: localId,
0124 |         createdAt: new Date().toISOString(),
0125 |       };
0126 |       setBusinesses((prev) => [newBiz, ...prev]);
0127 |       return newBiz;
0128 |     }
0129 |   };
0130 |
0131 |   const updateBusiness = async (id: string, updates: Partial<Business>) => {
0132 |     setBusinesses((prev) =>
0133 |       prev.map((b) => (b.id === id ? { ...b, ...updates, updatedAt: new Date().toISOString()...
0134 |     );
0135 |     try {
0136 |       await businessesApi.update(id, {
0137 |         name: updates.name,
0138 |         industry: updates.industry,
0139 |         email: updates.contactEmail,
0140 |         phone: updates.contactPhone,
0141 |         is_active: updates.status === 'active',
0142 |       });
0143 |     } catch (err) {
0144 |       console.warn('[BusinessProvider] Backend update error:', err);
0145 |     }
0146 |   };
0147 |
0148 |   const deleteBusiness = async (id: string) => {
0149 |     setBusinesses((prev) => prev.filter((b) => b.id !== id));
0150 |     if (selectedBusinessId === id) {
0151 |       setSelectedBusinessId('all');
0152 |     }
0153 |     try {
0154 |       await businessesApi.delete(id);
0155 |     } catch (err) {
0156 |       console.warn('[BusinessProvider] Backend delete error:', err);
0157 |     }
0158 |   };
0159 |
0160 |   const assignAgentToBusiness = async (businessId: string, agentId: string) => {
0161 |     setBusinesses((prev) =>
```

PART I: SOURCE CODE DEPOSIT — FIRST TEN PAGES [PAGE 7 OF 10]

```
0162 |       prev.map((b) => {
0163 |         if (b.id === businessId) {
0164 |           if (!b.assignedAgentIds.includes(agentId)) {
0165 |             return { ...b, assignedAgentIds: [...b.assignedAgentIds, agentId] };
0166 |           }
0167 |         }
0168 |         return b;
0169 |       })
0170 |     );
0171 |     try {
0172 |       await businessesApi.assignAgent(businessId, agentId);
0173 |     } catch (err) {
0174 |       console.warn('[BusinessProvider] Backend assign error:', err);
0175 |     }
0176 |   };
0177 |
0178 |   const unassignAgentFromBusiness = async (businessId: string, agentId: string) => {
0179 |     setBusinesses((prev) =>
0180 |       prev.map((b) => {
0181 |         if (b.id === businessId) {
0182 |           return {
0183 |             ...b,
0184 |             assignedAgentIds: b.assignedAgentIds.filter((id) => id !== agentId),
0185 |           };
0186 |         }
0187 |         return b;
0188 |       })
0189 |     );
0190 |     try {
0191 |       await businessesApi.unassignAgent(businessId, agentId);
0192 |     } catch (err) {
0193 |       console.warn('[BusinessProvider] Backend unassign error:', err);
0194 |     }
0195 |   };
0196 |
0197 |   const getBusinessForAgent = (agentId?: string | null): Business | undefined => {
0198 |     if (!agentId) return undefined;
0199 |     return businesses.find((b) => b.assignedAgentIds.includes(agentId));
0200 |   };
0201 |
0202 |   return (
0203 |     <BusinessContext.Provider
0204 |       value={{
0205 |         businesses,
0206 |         selectedBusinessId,
0207 |         selectedBusiness,
0208 |         isAllBusinesses,
0209 |         isLoading,
0210 |         refreshBusinesses,
0211 |         selectBusiness,
0212 |         addBusiness,
0213 |         updateBusiness,
0214 |         deleteBusiness,
0215 |         assignAgentToBusiness,
0216 |         unassignAgentFromBusiness,
0217 |         getBusinessForAgent,
0218 |       }}
0219 |     >
0220 |       {children}
0221 |     </BusinessContext.Provider>
0222 |   );
0223 | };
0224 |
0225 | export const useBusiness = () => {
0226 |   const context = useContext(BusinessContext);
0227 |   if (!context) {
```

PART I: SOURCE CODE DEPOSIT — FIRST TEN PAGES [PAGE 8 OF 10]

```
0228 |     throw new Error('useBusiness must be used within a BusinessProvider');
0229 |   }
0230 |   return context;
0231 | };


// ============================================================================
// FILE IDENTIFIER: src/app/providers/ToastProvider.tsx
// STATUTORY CLASS: Computer Programme / Literary Work (Section 2(o))
// ============================================================================
0001 | import React, { createContext, useContext, useState, useCallback } from 'react';
0002 | import { CheckCircle2, AlertCircle, AlertTriangle, Info, X } from 'lucide-react';
0003 |
0004 | export type ToastType = 'success' | 'error' | 'warning' | 'info';
0005 |
0006 | export interface ToastItem {
0007 |   id: string;
0008 |   type: ToastType;
0009 |   title: string;
0010 |   message?: string;
0011 |   duration?: number;
0012 | }
0013 |
0014 | interface ToastContextType {
0015 |   toasts: ToastItem[];
0016 |   addToast: (toast: Omit<ToastItem, 'id'>) => void;
0017 |   removeToast: (id: string) => void;
0018 |   success: (title: string, message?: string) => void;
0019 |   error: (title: string, message?: string) => void;
0020 |   warning: (title: string, message?: string) => void;
0021 |   info: (title: string, message?: string) => void;
0022 | }
0023 |
0024 | const ToastContext = createContext<ToastContextType | undefined>(undefined);
0025 |
0026 | export const ToastProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
0027 |   const [toasts, setToasts] = useState<ToastItem[]>([]);
0028 |
0029 |   const removeToast = useCallback((id: string) => {
0030 |     setToasts((prev) => prev.filter((t) => t.id !== id));
0031 |   }, []);
0032 |
0033 |   const addToast = useCallback(
0034 |     ({ type, title, message, duration = 4000 }: Omit<ToastItem, 'id'>) => {
0035 |       const id = Math.random().toString(36).substring(2, 9);
0036 |       setToasts((prev) => [...prev, { id, type, title, message, duration }]);
0037 |
0038 |       if (duration > 0) {
0039 |         setTimeout(() => {
0040 |           removeToast(id);
0041 |         }, duration);
0042 |       }
0043 |     },
0044 |     [removeToast]
0045 |   );
0046 |
0047 |   const success = useCallback((title: string, message?: string) => addToast({ type: 'success...
0048 |   const error = useCallback((title: string, message?: string) => addToast({ type: 'error', t...
0049 |   const warning = useCallback((title: string, message?: string) => addToast({ type: 'warning...
0050 |   const info = useCallback((title: string, message?: string) => addToast({ type: 'info', tit...
0051 |
0052 |   const getIcon = (type: ToastType) => {
0053 |     switch (type) {
0054 |       case 'success':
0055 |         return <CheckCircle2 className="w-5 h-5 text-emerald-500 shrink-0" />;
0056 |       case 'error':
```

PART I: SOURCE CODE DEPOSIT — FIRST TEN PAGES [PAGE 9 OF 10]

```
0057 |         return <AlertCircle className="w-5 h-5 text-rose-500 shrink-0" />;
0058 |       case 'warning':
0059 |         return <AlertTriangle className="w-5 h-5 text-amber-500 shrink-0" />;
0060 |       case 'info':
0061 |         return <Info className="w-5 h-5 text-orange-400 shrink-0" />;
0062 |     }
0063 |   };
0064 |
0065 |   return (
0066 |     <ToastContext.Provider value={{ toasts, addToast, removeToast, success, error, warning, ...
0067 |       {children}
0068 |       <div className="fixed bottom-5 right-5 z-50 flex flex-col gap-2.5 max-w-md w-full poin...
0069 |         {toasts.map((toast) => (
0070 |           <div
0071 |             key={toast.id}
0072 |             className="pointer-events-auto flex items-start gap-3 p-4 rounded-2xl border sha...
0073 |               bg-white/90 dark:bg-[var(--modal-glass-bg)] backdrop-blur-[20px] saturate-[140%]
0074 |               border-slate-200/80 dark:border-white/[0.08]"
0075 |           >
0076 |             {getIcon(toast.type)}
0077 |             <div className="flex-1 min-w-0">
0078 |               <h4 className="text-sm font-semibold text-slate-900 dark:text-white">{toast.ti...
0079 |               {toast.message && (
0080 |                 <p className="text-xs text-slate-600 dark:text-slate-400 mt-0.5 leading-rela...
0081 |               )}
0082 |             </div>
0083 |             <button
0084 |               onClick={() => removeToast(toast.id)}
0085 |               className="text-slate-400 hover:text-slate-600 dark:hover:text-slate-200 p-1 r...
0086 |             >
0087 |               <X className="w-4 h-4" />
0088 |             </button>
0089 |           </div>
0090 |         ))}
0091 |       </div>
0092 |     </ToastContext.Provider>
0093 |   );
0094 | };
0095 |
0096 | export const useToast = (): ToastContextType => {
0097 |   const context = useContext(ToastContext);
0098 |   if (!context) throw new Error('useToast must be used within ToastProvider');
0099 |   return context;
0100 | };
```

```
// ============================================================================
// FILE IDENTIFIER: src/app/providers/ThemeProvider.tsx
// STATUTORY CLASS: Computer Programme / Literary Work (Section 2(o))
// ============================================================================
0001 | import React, { createContext, useContext, useEffect, useState } from 'react';
0002 |
0003 | type Theme = 'light' | 'dark' | 'system';
0004 |
0005 | /** Interface "look": 'classic' is the original glass/dark design system;
0006 |  * 'modern' is the clean light design (solid white cards, soft shadows,
0007 |  * pill controls — see styles/modern.css). The look is applied as
0008 |  * data-look on <html>; modern always resolves to the light scheme. */
0009 | export type Look = 'classic' | 'modern';
0010 |
0011 | interface ThemeContextType {
0012 |   theme: Theme;
0013 |   resolvedTheme: 'light' | 'dark';
0014 |   setTheme: (theme: Theme) => void;
0015 |   toggleTheme: () => void;
0016 |   look: Look;
```

PART I: SOURCE CODE DEPOSIT — FIRST TEN PAGES [PAGE 10 OF 10]

```
0017 |   setLook: (look: Look) => void;
0018 | }
0019 |
0020 | const ThemeContext = createContext<ThemeContextType | undefined>(undefined);
0021 |
0022 | export const ThemeProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
0023 |   const [theme, setTheme] = useState<Theme>(() => {
0024 |     const saved = localStorage.getItem('voxora_theme') as Theme | null;
0025 |     return saved || 'dark'; // default to modern dark theme
0026 |   });
0027 |
0028 |   const [look, setLook] = useState<Look>(() => {
0029 |     const saved = localStorage.getItem('voxora_look') as Look | null;
0030 |     return saved === 'modern' ? 'modern' : 'classic';
0031 |   });
0032 |
0033 |   const [resolvedTheme, setResolvedTheme] = useState<'light' | 'dark'>('dark');
0034 |
0035 |   useEffect(() => {
0036 |     const root = document.documentElement;
0037 |     const mediaQuery = window.matchMedia('(prefers-color-scheme: dark)');
0038 |
0039 |     root.dataset.look = look;
0040 |     localStorage.setItem('voxora_look', look);
0041 |
0042 |     const updateResolved = () => {
0043 |       let isDark = false;
0044 |       if (look === 'modern') {
0045 |         isDark = false; // the modern look is a light design, always
0046 |       } else if (theme === 'system') {
0047 |         isDark = mediaQuery.matches;
0048 |       } else {
0049 |         isDark = theme === 'dark';
0050 |       }
0051 |
0052 |       setResolvedTheme(isDark ? 'dark' : 'light');
0053 |       if (isDark) {
0054 |         root.classList.add('dark');
0055 |       } else {
0056 |         root.classList.remove('dark');
0057 |       }
0058 |     };
0059 |
0060 |     updateResolved();
0061 |     localStorage.setItem('voxora_theme', theme);
0062 |
0063 |     const listener = () => {
0064 |       if (theme === 'system') updateResolved();
0065 |     };
0066 |
0067 |     mediaQuery.addEventListener('change', listener);
0068 |     return () => mediaQuery.removeEventListener('change', listener);
0069 |   }, [theme, look]);
0070 |
0071 |   const toggleTheme = () => {
0072 |     setTheme((prev) => (prev === 'dark' ? 'light' : 'dark'));
0073 |   };
0074 |
0075 |   return (
0076 |     <ThemeContext.Provider value={{ theme, resolvedTheme, setTheme, toggleTheme, look, setLo...
0077 |       {children}
0078 |     </ThemeContext.Provider>
0079 |   );
0080 | };
0081 |
0082 | export const useTheme = (): ThemeContextType => {
```

STATUTORY NOTICE OF INTERMEDIATE REPOSITORY DEPOSIT

VOLUMINOUS SOFTWARE INTERMEDIATE OMISSION

OFFICIAL NOTICE UNDER RULE 70(5), COPYRIGHT RULES, 2013:
In accordance with the statutory deposit procedure sanctioned by the Indian Copyright Office for voluminous computer software exceeding 20 pages, the intermediate source code files of the application comprising **105 source files** and **22,049 lines of code** are omitted from this printed deposit document. All omitted files are recorded, categorized, and cryptographically verified in the accompanying SOURCE_CODE_INVENTORY.xlsx.

SUMMARY OF INTERMEDIATE REPOSITORY MODULES (OMITTED PER RULE 70(5))

Subsystem / Module Directory	Files	Lines	Core Functional Modules
src/features/agents/	3	2,025	AI Voice Agent Studio, Prompt Builder Drawer, Agent Dossier Export.
src/features/calls/	4	829	Active Telephony Monitor, Call Detail Drawer, Audio Stream Player.
src/features/leads/	1	398	Gemini Lead Intelligence Pipeline, Why-Interested Reasons, Excel Export.
src/features/dashboard/	2	1,168	Telephony Telemetry Dashboard, KPI Metrics, Recharts Visualizations.
src/features/questionnaires/	4	1,280	Interactive Branching Questionnaire Flow Editor, Excel/Docx Importers.
src/features/modern/	17	2,649	Futuristic Modern Design UI Suite (17 Dedicated Modern Feature Views).
src/features/settings/	5	2,014	Sarvam AI, VoBiz Carrier, Twilio Carrier, and Gemini Settings Consoles.
src/features/model_lab/	1	2,066	Voice Latency Testing Workbench, STT/TTS Diagnostic Benchmark Suite.
src/components/ui/	21	2,184	Custom UI Component Library (Buttons, Cards, Modals, AudioPlayer, Tabs).
src/services/api/	17	1,264	RESTful HTTP and WebSocket API Client Service Layer (17 Domain Services).

The Applicant undertakes to produce the unredacted source code of any or all omitted intermediate files upon any specific requisition issued by the Registrar of Copyrights during statutory scrutiny.

PART II: SOURCE CODE DEPOSIT — LAST TEN PAGES [PAGE 1 OF 10]

```
// ============================================================================
// FILE IDENTIFIER: src/utils/quickDial.ts
// STATUTORY CLASS: Computer Programme / Literary Work (Section 2(o))
// ============================================================================
0001 | export const triggerQuickDial = (agentId?: string) => {
0002 |   window.dispatchEvent(new CustomEvent('voxora-quick-dial', { detail: { agentId } }));
0003 | };


// ============================================================================
// FILE IDENTIFIER: src/utils/excelExport.ts
// STATUTORY CLASS: Computer Programme / Literary Work (Section 2(o))
// ============================================================================
0001 | import * as XLSX from 'xlsx';
0002 | import { isAfter, subDays, startOfDay } from 'date-fns';
0003 |
0004 | export type TimeFilterRange = 'all' | 'day' | 'week' | 'month';
0005 |
0006 | /**
0007 |  * Filter an array of items by a date field using the chosen time filter range.
0008 |  * - 'day': items from the start of today (or last 24h)
0009 |  * - 'week': items within the last 7 days
0010 |  * - 'month': items within the last 30 days
0011 |  * - 'all': all items without date restriction
0012 |  */
0013 | export function filterByDateRange<T>(
0014 |   items: T[],
0015 |   getDateStr: (item: T) => string | Date | null | undefined,
0016 |   range: TimeFilterRange
0017 | ): T[] {
0018 |   if (range === 'all') return items;
0019 |
0020 |   const now = new Date();
0021 |   let thresholdDate: Date;
0022 |
0023 |   if (range === 'day') {
0024 |     thresholdDate = startOfDay(now);
0025 |   } else if (range === 'week') {
0026 |     thresholdDate = subDays(now, 7);
0027 |   } else if (range === 'month') {
0028 |     thresholdDate = subDays(now, 30);
0029 |   } else {
0030 |     return items;
0031 |   }
0032 |
0033 |   return items.filter((item) => {
0034 |     const raw = getDateStr(item);
0035 |     if (!raw) return false;
0036 |     try {
0037 |       const dt = typeof raw === 'string' ? new Date(raw) : raw;
0038 |       if (isNaN(dt.getTime())) return false;
0039 |       return isAfter(dt, thresholdDate) || dt.getTime() === thresholdDate.getTime();
0040 |     } catch {
0041 |       return false;
0042 |     }
0043 |   });
0044 | }
0045 |
0046 | /**
0047 |  * Exports JSON data rows directly to an Excel (.xlsx) spreadsheet with auto-fitted column w...
0048 |  */
0049 | export function exportToExcel(
0050 |   data: Record<string, any>[],
0051 |   fileName: string,
0052 |   sheetName: string = 'Data'
0053 | ): void {
```

PART II: SOURCE CODE DEPOSIT — LAST TEN PAGES [PAGE 2 OF 10]

```
0054 |   if (!data || data.length === 0) {
0055 |     alert('No data available to export for the selected filter.');
0056 |     return;
0057 |   }
0058 |
0059 |   const ws = XLSX.utils.json_to_sheet(data);
0060 |
0061 |   // Auto-calculate column widths
0062 |   if (data.length > 0) {
0063 |     const keys = Object.keys(data[0]);
0064 |     const colWidths = keys.map((key) => {
0065 |       let maxLen = key.length;
0066 |       data.forEach((row) => {
0067 |         const val = row[key];
0068 |         if (val !== null && val !== undefined) {
0069 |           const str = String(val);
0070 |           if (str.length > maxLen) {
0071 |             maxLen = Math.min(str.length, 50); // cap max width at 50
0072 |           }
0073 |         }
0074 |       });
0075 |       return { wch: Math.max(maxLen + 3, 12) };
0076 |     });
0077 |     ws['!cols'] = colWidths;
0078 |   }
0079 |
0080 |   const wb = XLSX.utils.book_new();
0081 |   XLSX.utils.book_append_sheet(wb, ws, sheetName);
0082 |
0083 |   const cleanName = fileName.endsWith('.xlsx') ? fileName : `${fileName}.xlsx`;
0084 |   XLSX.writeFile(wb, cleanName);
0085 | }


// ============================================================================
// FILE IDENTIFIER: src/utils/agentPdfExport.ts
// STATUTORY CLASS: Computer Programme / Literary Work (Section 2(o))
// ============================================================================
0001 | import { jsPDF } from 'jspdf';
0002 | import autoTable from 'jspdf-autotable';
0003 | import { format } from 'date-fns';
0004 | import { AgentDocument, AgentDashboardResponse } from '../types/api';
0005 |
0006 | /**
0007 |  * Generates and downloads a multi-page, executive-grade PDF report
0008 |  * of an Agent's Dashboard containing complete performance metrics,
0009 |  * telephony configuration, lead conversions, and recent call telemetry logs.
0010 |  */
0011 | export function generateAgentDashboardPdf(
0012 |   agent: AgentDocument,
0013 |   dashboard: AgentDashboardResponse,
0014 |   businessName?: string
0015 | ): void {
0016 |   const doc = new jsPDF({
0017 |     orientation: 'portrait',
0018 |     unit: 'pt',
0019 |     format: 'a4',
0020 |   });
0021 |
0022 |   const pageWidth = doc.internal.pageSize.getWidth();
0023 |   const pageHeight = doc.internal.pageSize.getHeight();
0024 |   const margin = 40;
0025 |   const contentWidth = pageWidth - margin * 2;
0026 |
0027 |   // ------------------------------------------------------------
0028 |   // 1. Header Banner
```

PART II: SOURCE CODE DEPOSIT — LAST TEN PAGES [PAGE 3 OF 10]

```
0029 |   // ------------------------------------------------------------
0030 |   doc.setFillColor(15, 23, 42); // slate-900
0031 |   doc.rect(0, 0, pageWidth, 85, 'F');
0032 |
0033 |   // Accent bar
0034 |   doc.setFillColor(249, 115, 22); // orange-500
0035 |   doc.rect(0, 80, pageWidth, 5, 'F');
0036 |
0037 |   doc.setTextColor(255, 255, 255);
0038 |   doc.setFont('helvetica', 'bold');
0039 |   doc.setFontSize(18);
0040 |   doc.text('ARB SOFTTECH · AGENT DASHBOARD TELEMETRY', margin, 36);
0041 |
0042 |   doc.setFont('helvetica', 'normal');
0043 |   doc.setFontSize(9);
0044 |   doc.setTextColor(203, 213, 225); // slate-300
0045 |   doc.text('Executive Intelligence & Voice Agent Performance Report', margin, 52);
0046 |
0047 |   const reportDateStr = format(new Date(), 'MMMM dd, yyyy · HH:mm:ss');
0048 |   doc.setFontSize(8);
0049 |   doc.setTextColor(148, 163, 184); // slate-400
0050 |   doc.text(`Generated: ${reportDateStr}`, pageWidth - margin, 52, { align: 'right' });
0051 |
0052 |   let y = 110;
0053 |
0054 |   // ------------------------------------------------------------
0055 |   // 2. Agent Profile Summary Card
0056 |   // ------------------------------------------------------------
0057 |   doc.setFillColor(248, 250, 252); // slate-50
0058 |   doc.setDrawColor(226, 232, 240); // slate-200
0059 |   doc.roundedRect(margin, y, contentWidth, 90, 6, 6, 'FD');
0060 |
0061 |   doc.setTextColor(15, 23, 42);
0062 |   doc.setFont('helvetica', 'bold');
0063 |   doc.setFontSize(14);
0064 |   doc.text(agent.name, margin + 14, y + 24);
0065 |
0066 |   // Status Badge
0067 |   const statusStr = agent.is_enabled ? 'ACTIVE' : 'INACTIVE';
0068 |   doc.setFillColor(agent.is_enabled ? 16 : 244, agent.is_enabled ? 185 : 63, agent.is_enable...
0069 |   doc.roundedRect(margin + contentWidth - 75, y + 12, 60, 16, 3, 3, 'F');
0070 |   doc.setTextColor(255, 255, 255);
0071 |   doc.setFont('helvetica', 'bold');
0072 |   doc.setFontSize(8);
0073 |   doc.text(statusStr, margin + contentWidth - 45, y + 23, { align: 'center' });
0074 |
0075 |   doc.setFont('helvetica', 'normal');
0076 |   doc.setFontSize(8.5);
0077 |   doc.setTextColor(71, 85, 105);
0078 |
0079 |   const col1X = margin + 14;
0080 |   const col2X = margin + 265;
0081 |
0082 |   // Row 1: Agent ID & Provider
0083 |   doc.text(`Agent ID: ${agent.agent_id}`, col1X, y + 44);
0084 |   doc.text(`Provider: ${(agent.provider || 'vobiz').toUpperCase()}`, col2X, y + 44);
0085 |
0086 |   // Row 2: Assigned Business & Language
0087 |   doc.text(`Assigned Business: ${businessName || 'Default Account'}`, col1X, y + 59);
0088 |   doc.text(`Language: ${agent.default_language || 'Marathi'}`, col2X, y + 59);
0089 |
0090 |   // Row 3: Concurrency Limit & Trigger Path
0091 |   doc.text(`Concurrency Limit: ${agent.concurrency || 2}x Channels`, col1X, y + 74);
0092 |   doc.text(`Trigger Path: ${(agent as any).dograh_trigger_path || '/voice'}`, col2X, y + 74);
0093 |
0094 |   y += 108;
```

PART II: SOURCE CODE DEPOSIT — LAST TEN PAGES [PAGE 4 OF 10]

```
0095 |
0096 |   // ------------------------------------------------------------
0097 |   // 3. Operational Performance KPI Cards
0098 |   // ------------------------------------------------------------
0099 |   doc.setTextColor(15, 23, 42);
0100 |   doc.setFont('helvetica', 'bold');
0101 |   doc.setFontSize(11);
0102 |   doc.text('Key Operational KPIs', margin, y + 10);
0103 |   y += 18;
0104 |
0105 |   const totalCalls = dashboard.total_calls || 0;
0106 |   const completedCalls = dashboard.completed_calls || 0;
0107 |   const completionRate = totalCalls > 0 ? Math.round((completedCalls / totalCalls) * 100) : 0;
0108 |   const avgDuration = (dashboard.average_duration_seconds || 0).toFixed(1);
0109 |   const totalVoiceSecs = (dashboard as any).total_duration_seconds ?? (completedCalls * (das...
0110 |   const totalMinutes = (totalVoiceSecs / 60).toFixed(1);
0111 |
0112 |   const kpis = [
0113 |     { label: 'Total Calls', value: `${totalCalls}`, sub: 'Telemetry history' },
0114 |     { label: 'Completed Calls', value: `${completedCalls}`, sub: `${completionRate}% Success...
0115 |     { label: 'Avg Duration', value: `${avgDuration}s`, sub: 'Per connected session' },
0116 |     { label: 'Total Minutes', value: `${totalMinutes}m`, sub: 'Voice time consumed' },
0117 |   ];
0118 |
0119 |   const cardWidth = (contentWidth - 30) / 4;
0120 |   kpis.forEach((kpi, idx) => {
0121 |     const cardX = margin + idx * (cardWidth + 10);
0122 |     doc.setFillColor(255, 255, 255);
0123 |     doc.setDrawColor(226, 232, 240);
0124 |     doc.roundedRect(cardX, y, cardWidth, 54, 4, 4, 'FD');
0125 |
0126 |     doc.setFont('helvetica', 'normal');
0127 |     doc.setFontSize(8);
0128 |     doc.setTextColor(100, 116, 139);
0129 |     doc.text(kpi.label, cardX + 8, y + 15);
0130 |
0131 |     doc.setFont('helvetica', 'bold');
0132 |     doc.setFontSize(13);
0133 |     doc.setTextColor(15, 23, 42);
0134 |     doc.text(kpi.value, cardX + 8, y + 33);
0135 |
0136 |     doc.setFont('helvetica', 'normal');
0137 |     doc.setFontSize(7);
0138 |     doc.setTextColor(148, 163, 184);
0139 |     doc.text(kpi.sub, cardX + 8, y + 46);
0140 |   });
0141 |
0142 |   y += 68;
0143 |
0144 |   // ------------------------------------------------------------
0145 |   // 4. Lead Intelligence & Conversion Breakdown
0146 |   // ------------------------------------------------------------
0147 |   doc.setTextColor(15, 23, 42);
0148 |   doc.setFont('helvetica', 'bold');
0149 |   doc.setFontSize(11);
0150 |   doc.text('Customer Intent & Lead Intelligence Classification', margin, y + 10);
0151 |   y += 18;
0152 |
0153 |   const interestedCount = dashboard.interested_count || 0;
0154 |   const notInterestedCount = dashboard.not_interested_count || 0;
0155 |   const uncertainCount = dashboard.uncertain_count || 0;
0156 |   const conversionRate = completedCalls > 0 ? Math.round((interestedCount / completedCalls) ...
0157 |
0158 |   const intentKpis = [
0159 |     { label: 'Interested Leads', value: `${interestedCount}`, color: [16, 185, 129] },
0160 |     { label: 'Not Interested', value: `${notInterestedCount}`, color: [244, 63, 94] },
```

PART II: SOURCE CODE DEPOSIT — LAST TEN PAGES [PAGE 5 OF 10]

```
    { label: 'Uncertain / Pending', value: `${uncertainCount}`, color: [245, 158, 11] },
    { label: 'Conversion Ratio', value: `${conversionRate}%`, color: [99, 102, 241] },
  ];

  intentKpis.forEach((item, idx) => {
    const cardX = margin + idx * (cardWidth + 10);
    doc.setFillColor(248, 250, 252);
    doc.setDrawColor(226, 232, 240);
    doc.roundedRect(cardX, y, cardWidth, 48, 4, 4, 'FD');

    // Colored left accent bar
    doc.setFillColor(item.color[0], item.color[1], item.color[2]);
    doc.rect(cardX, y, 4, 48, 'F');

    doc.setFont('helvetica', 'normal');
    doc.setFontSize(8);
    doc.setTextColor(100, 116, 139);
    doc.text(item.label, cardX + 10, y + 16);

    doc.setFont('helvetica', 'bold');
    doc.setFontSize(13);
    doc.setTextColor(15, 23, 42);
    doc.text(item.value, cardX + 10, y + 36);
  });

  y += 62;

  // ------------------------------------------------------------
  // 5. System Prompt & Directive Preview
  // ------------------------------------------------------------
  if (agent.system_prompt) {
    doc.setTextColor(15, 23, 42);
    doc.setFont('helvetica', 'bold');
    doc.setFontSize(11);
    doc.text('Voice Persona & Directive Prompt', margin, y + 10);
    y += 18;

    doc.setFillColor(250, 250, 250);
    doc.setDrawColor(226, 232, 240);

    const promptText = agent.system_prompt.trim();
    const promptLines = doc.splitTextToSize(promptText, contentWidth - 20).slice(0, 4);
    const boxHeight = promptLines.length * 13 + 16;

    doc.roundedRect(margin, y, contentWidth, boxHeight, 4, 4, 'FD');
    doc.setFont('helvetica', 'normal');
    doc.setFontSize(8);
    doc.setTextColor(71, 85, 105);
    doc.text(promptLines, margin + 10, y + 16);

    y += boxHeight + 16;
  }

  // ------------------------------------------------------------
  // 6. Recent Telemetry Calls Table
  // ------------------------------------------------------------
  doc.setTextColor(15, 23, 42);
  doc.setFont('helvetica', 'bold');
  doc.setFontSize(11);
  doc.text('Recent Call Telemetry Logs', margin, y + 10);
  y += 16;

  const recentCalls = dashboard.recent_calls || [];
  const tableData = recentCalls.length > 0
    ? recentCalls.map((call, idx) => [
        `#${idx + 1}`,
```

PART II: SOURCE CODE DEPOSIT — LAST TEN PAGES [PAGE 6 OF 10]

```
0227 |         call.phone_number || 'N/A',
0228 |         (call.state || 'COMPLETED').toUpperCase(),
0229 |         `${call.duration ? call.duration.toFixed(1) : '0.0'}s`,
0230 |         call.provider ? call.provider.toUpperCase() : 'VOBIZ',
0231 |         call.created_at ? format(new Date(call.created_at), 'yyyy-MM-dd HH:mm') : 'N/A',
0232 |       ])
0233 |     : [
0234 |         ['—', 'No recent calls recorded', '—', '—', '—', '—'],
0235 |       ];
0236 |
0237 |   autoTable(doc, {
0238 |     startY: y,
0239 |     head: [['#', 'Phone Number', 'Call State', 'Duration', 'Provider', 'Timestamp']],
0240 |     body: tableData,
0241 |     margin: { left: margin, right: margin },
0242 |     styles: {
0243 |       fontSize: 8,
0244 |       cellPadding: 5,
0245 |       textColor: [51, 65, 85],
0246 |       font: 'helvetica',
0247 |     },
0248 |     headStyles: {
0249 |       fillColor: [15, 23, 42],
0250 |       textColor: [255, 255, 255],
0251 |       fontStyle: 'bold',
0252 |       fontSize: 8,
0253 |     },
0254 |     alternateRowStyles: {
0255 |       fillColor: [248, 250, 252],
0256 |     },
0257 |     didDrawPage: (data) => {
0258 |       // Footer on all pages
0259 |       const str = `Page ${data.pageNumber}`;
0260 |       doc.setFontSize(8);
0261 |       doc.setTextColor(148, 163, 184);
0262 |       doc.text(str, pageWidth - margin, pageHeight - 20, { align: 'right' });
0263 |       doc.text('ARB SoftTech AI Voice Platform · Strictly Confidential', margin, pageHeight ...
0264 |     },
0265 |   });
0266 |
0267 |   // Save the PDF
0268 |   const cleanName = agent.name.toLowerCase().replace(/[^a-z0-9]+/g, '_');
0269 |   const dateStamp = format(new Date(), 'yyyyMMdd');
0270 |   doc.save(`${cleanName}_dashboard_${dateStamp}.pdf`);
0271 | }
```

```
// ============================================================================
// FILE IDENTIFIER: src/utils/businessPdfExport.ts
// STATUTORY CLASS: Computer Programme / Literary Work (Section 2(o))
// ============================================================================
0001 | import { jsPDF } from 'jspdf';
0002 | import autoTable from 'jspdf-autotable';
0003 | import { format } from 'date-fns';
0004 | import { Business } from '../app/providers/BusinessProvider';
0005 | import { AgentDocument, CallDocument } from '../types/api';
0006 |
0007 | export interface BusinessPdfData {
0008 |   business: Business;
0009 |   assignedAgents: AgentDocument[];
0010 |   stats?: {
0011 |     total_calls?: number;
0012 |     completed_calls?: number;
0013 |     failed_calls?: number;
0014 |     active_calls?: number;
0015 |     queued_calls?: number;
```

PART II: SOURCE CODE DEPOSIT — LAST TEN PAGES [PAGE 7 OF 10]

```
0016 |     average_duration_seconds?: number;
0017 |     total_duration_seconds?: number;
0018 |     total_leads?: number;
0019 |     qualified_leads?: number;
0020 |     interested_count?: number;
0021 |     not_interested_count?: number;
0022 |     uncertain_count?: number;
0023 |   };
0024 |   recentCalls?: CallDocument[];
0025 | }
0026 |
0027 | /**
0028 |  * Generates and downloads an executive-grade PDF telemetry report
0029 |  * for a Client Business Dashboard containing operational metrics,
0030 |  * AI agent fleet status, lead conversions, and recent call activity.
0031 |  */
0032 | export function generateBusinessDashboardPdf(data: BusinessPdfData): void {
0033 |   const { business, assignedAgents, stats = {}, recentCalls = [] } = data;
0034 |
0035 |   const doc = new jsPDF({
0036 |     orientation: 'portrait',
0037 |     unit: 'pt',
0038 |     format: 'a4',
0039 |   });
0040 |
0041 |   const pageWidth = doc.internal.pageSize.getWidth();
0042 |   const pageHeight = doc.internal.pageSize.getHeight();
0043 |   const margin = 40;
0044 |   const contentWidth = pageWidth - margin * 2;
0045 |
0046 |   // ------------------------------------------------------------
0047 |   // 1. Header Banner
0048 |   // ------------------------------------------------------------
0049 |   doc.setFillColor(15, 23, 42); // slate-900
0050 |   doc.rect(0, 0, pageWidth, 85, 'F');
0051 |
0052 |   // Emerald accent bar
0053 |   doc.setFillColor(16, 185, 129); // emerald-500
0054 |   doc.rect(0, 80, pageWidth, 5, 'F');
0055 |
0056 |   doc.setTextColor(255, 255, 255);
0057 |   doc.setFont('helvetica', 'bold');
0058 |   doc.setFontSize(17);
0059 |   doc.text('ARB SOFTTECH · CLIENT BUSINESS TELEMETRY', margin, 36);
0060 |
0061 |   doc.setFont('helvetica', 'normal');
0062 |   doc.setFontSize(9);
0063 |   doc.setTextColor(203, 213, 225); // slate-300
0064 |   doc.text('Client Account Operational Performance & AI Telemetry Audit', margin, 52);
0065 |
0066 |   const reportDateStr = format(new Date(), 'MMMM dd, yyyy · HH:mm:ss');
0067 |   doc.setFontSize(8);
0068 |   doc.setTextColor(148, 163, 184); // slate-400
0069 |   doc.text(`Generated: ${reportDateStr}`, pageWidth - margin, 52, { align: 'right' });
0070 |
0071 |   let y = 108;
0072 |
0073 |   // ------------------------------------------------------------
0074 |   // 2. Client Business Profile Summary Card
0075 |   // ------------------------------------------------------------
0076 |   doc.setFillColor(248, 250, 252); // slate-50
0077 |   doc.setDrawColor(226, 232, 240); // slate-200
0078 |   doc.roundedRect(margin, y, contentWidth, 92, 6, 6, 'FD');
0079 |
0080 |   doc.setTextColor(15, 23, 42);
0081 |   doc.setFont('helvetica', 'bold');
```

PART II: SOURCE CODE DEPOSIT — LAST TEN PAGES [PAGE 8 OF 10]

```
0082 |   doc.setFontSize(14);
0083 |   doc.text(business.name, margin + 14, y + 24);
0084 |
0085 |   // Status Badge
0086 |   const statusStr = (business.status || 'ACTIVE').toUpperCase();
0087 |   const isAct = business.status === 'active';
0088 |   doc.setFillColor(isAct ? 16 : 245, isAct ? 185 : 158, isAct ? 129 : 11);
0089 |   doc.roundedRect(margin + contentWidth - 85, y + 12, 70, 16, 3, 3, 'F');
0090 |   doc.setTextColor(255, 255, 255);
0091 |   doc.setFont('helvetica', 'bold');
0092 |   doc.setFontSize(8);
0093 |   doc.text(statusStr, margin + contentWidth - 50, y + 23, { align: 'center' });
0094 |
0095 |   doc.setFont('helvetica', 'normal');
0096 |   doc.setFontSize(8.5);
0097 |   doc.setTextColor(71, 85, 105); // slate-600
0098 |
0099 |   const col1X = margin + 14;
0100 |   const col2X = margin + 265;
0101 |
0102 |   const createdDateStr = business.createdAt
0103 |     ? format(new Date(business.createdAt), 'MMM dd, yyyy')
0104 |     : 'Registered Account';
0105 |
0106 |   // Row 1: Account ID & Contact Phone
0107 |   doc.text(`Account ID: ${business.id}`, col1X, y + 44);
0108 |   doc.text(`Contact Phone: ${business.contactPhone || 'N/A'}`, col2X, y + 44);
0109 |
0110 |   // Row 2: Industry & Assigned Agents
0111 |   doc.text(`Industry: ${business.industry || 'General Enterprise'}`, col1X, y + 59);
0112 |   doc.text(`Assigned Agents: ${assignedAgents.length} Active AI Agent(s)`, col2X, y + 59);
0113 |
0114 |   // Row 3: Contact Email & Client Since
0115 |   doc.text(`Contact Email: ${business.contactEmail || 'N/A'}`, col1X, y + 74);
0116 |   doc.text(`Client Since: ${createdDateStr}`, col2X, y + 74);
0117 |
0118 |   y += 108;
0119 |
0120 |   // ------------------------------------------------------------
0121 |   // 3. Operational Performance KPI Cards
0122 |   // ------------------------------------------------------------
0123 |   doc.setTextColor(15, 23, 42);
0124 |   doc.setFont('helvetica', 'bold');
0125 |   doc.setFontSize(11);
0126 |   doc.text('Key Operational KPIs', margin, y + 10);
0127 |   y += 18;
0128 |
0129 |   const totalCalls = stats.total_calls || 0;
0130 |   const completedCalls = stats.completed_calls || 0;
0131 |   const completionRate = totalCalls > 0 ? Math.round((completedCalls / totalCalls) * 100) : 0;
0132 |   const totalLeads = stats.total_leads || 0;
0133 |   const qualLeads = stats.qualified_leads || 0;
0134 |   const conversionRate = totalLeads > 0 ? Math.round((qualLeads / totalLeads) * 100) : 0;
0135 |
0136 |   const avgDuration = (stats.average_duration_seconds || 0).toFixed(1);
0137 |   const totalVoiceSecs = stats.total_duration_seconds ?? (completedCalls * (stats.average_du...
0138 |   const totalMinutes = (totalVoiceSecs / 60).toFixed(1);
0139 |
0140 |   const kpis = [
0141 |     { label: 'Total Calls', value: `${totalCalls}`, sub: 'Telephony history' },
0142 |     { label: 'Completed Sessions', value: `${completedCalls}`, sub: `${completionRate}% Succ...
0143 |     { label: 'Leads Captured', value: `${totalLeads}`, sub: `${qualLeads} Qualified` },
0144 |     { label: 'Conversion Rate', value: `${conversionRate}%`, sub: 'Lead efficiency' },
0145 |   ];
0146 |
0147 |   const cardWidth = (contentWidth - 30) / 4;
```

PART II: SOURCE CODE DEPOSIT — LAST TEN PAGES [PAGE 9 OF 10]

```
  kpis.forEach((kpi, idx) => {
    const cardX = margin + idx * (cardWidth + 10);
    doc.setFillColor(255, 255, 255);
    doc.setDrawColor(226, 232, 240);
    doc.roundedRect(cardX, y, cardWidth, 54, 4, 4, 'FD');

    doc.setFont('helvetica', 'normal');
    doc.setFontSize(7.5);
    doc.setTextColor(100, 116, 139);
    doc.text(kpi.label.toUpperCase(), cardX + 8, y + 14);

    doc.setFont('helvetica', 'bold');
    doc.setFontSize(14);
    doc.setTextColor(15, 23, 42);
    doc.text(kpi.value, cardX + 8, y + 33);

    doc.setFont('helvetica', 'normal');
    doc.setFontSize(7.5);
    doc.setTextColor(71, 85, 105);
    doc.text(kpi.sub, cardX + 8, y + 46);
  });

  y += 66;

  // ------------------------------------------------------------
  // 4. Secondary Telemetry & Lead Intelligence Breakdown
  // ------------------------------------------------------------
  doc.setFillColor(248, 250, 252);
  doc.setDrawColor(226, 232, 240);
  doc.roundedRect(margin, y, contentWidth, 56, 4, 4, 'FD');

  doc.setFont('helvetica', 'bold');
  doc.setFontSize(9);
  doc.setTextColor(15, 23, 42);
  doc.text('Voice Telemetry & Customer Intent Intelligence', margin + 12, y + 18);

  doc.setFont('helvetica', 'normal');
  doc.setFontSize(8.5);
  doc.setTextColor(71, 85, 105);

  const interestedCount = stats.interested_count ?? qualLeads;
  const notInterestedCount = stats.not_interested_count ?? 0;
  const uncertainCount = stats.uncertain_count ?? 0;

  const t1 = margin + 12;
  const t2 = margin + 140;
  const t3 = margin + 270;
  const t4 = margin + 400;

  doc.text(`Avg Duration: ${avgDuration}s`, t1, y + 38);
  doc.text(`Total Voice Time: ${totalMinutes} mins`, t2, y + 38);
  doc.text(`Interested: ${interestedCount} (${totalLeads > 0 ? Math.round((interestedCount /...
  doc.text(`Uncertain / Not Interested: ${uncertainCount} / ${notInterestedCount}`, t4, y + ...

  y += 70;

  // ------------------------------------------------------------
  // 5. Assigned Voice Agents Fleet Table
  // ------------------------------------------------------------
  doc.setFont('helvetica', 'bold');
  doc.setFontSize(11);
  doc.setTextColor(15, 23, 42);
  doc.text(`Assigned AI Agent Fleet (${assignedAgents.length})`, margin, y + 6);
  y += 14;

  const agentTableRows = assignedAgents.map((ag, index) => [
```

PART II: SOURCE CODE DEPOSIT — LAST TEN PAGES [PAGE 10 OF 10]

```
    (index + 1).toString(),
    ag.name,
    (ag.provider || 'vobiz').toUpperCase(),
    ag.default_language || 'Hindi',
    `${ag.concurrency || 1} Channel(s)`,
    ag.is_enabled ? 'Active' : 'Disabled',
  ]);

  if (agentTableRows.length === 0) {
    agentTableRows.push(['1', 'No dedicated agent assigned', 'VOBIZ', 'English', '1 Channel'...
  }

  autoTable(doc, {
    startY: y,
    head: [['#', 'Agent Name', 'Telephony Provider', 'Primary Language', 'Concurrency', 'Sta...
    body: agentTableRows,
    theme: 'grid',
    headStyles: {
      fillColor: [15, 23, 42],
      textColor: [255, 255, 255],
      fontStyle: 'bold',
      fontSize: 8,
    },
    styles: {
      fontSize: 8,
      cellPadding: 4.5,
      textColor: [30, 41, 59],
      overflow: 'linebreak',
    },
    alternateRowStyles: {
      fillColor: [248, 250, 252],
    },
    columnStyles: {
      0: { cellWidth: 22, halign: 'center' },
      1: { cellWidth: 155 },
      2: { cellWidth: 90 },
      3: { cellWidth: 90 },
      4: { cellWidth: 80 },
      5: { cellWidth: 75, halign: 'center' },
    },
    margin: { left: margin, right: margin },
  });

  y = (doc as any).lastAutoTable.finalY + 20;

  // ------------------------------------------------------------
  // 6. Recent Calls Telemetry Table (if present)
  // ------------------------------------------------------------
  if (recentCalls.length > 0) {
    // If not enough space on current page, add new page
    if (y > pageHeight - 160) {
      doc.addPage();
      y = 50;
    }

    doc.setFont('helvetica', 'bold');
    doc.setFontSize(11);
    doc.setTextColor(15, 23, 42);
    doc.text(`Recent Calls Telemetry (${recentCalls.length})`, margin, y + 6);
    y += 14;

    const callTableRows = recentCalls.slice(0, 15).map((call, index) => [
      (index + 1).toString(),
      call.phone_number || 'Unknown',
      (call.state || 'completed').toUpperCase(),
      `${(call.duration || 0).toFixed(1)}s`,
```

CERTIFICATE OF SOURCE CODE INTEGRITY & AUTHENTICITY

This is to formally certify that the source code deposited in this document represents the genuine, authentic, and unaltered computer software instructions authored for the frontend client of the **ARB SoftTech AI Enterprise Voice Calling Agent Platform**. The source code has been verified against the master electronic version control repository.

CRYPTOGRAPHIC HASH SUMMARY OF DEPOSITED SOURCE FILES

Relative File Path	Size	Lines	SHA-256 Cryptographic Checksum
src/main.tsx	780 B	29	`67537f1bbae5ad1eb580d803194fc2a497b854e1de1c3be779f98afb2977e77d`
src/app/App.tsx	816 B	27	`d070d62506ecaee38b3799b3ac93d3180c65242bbe5f6f4297895dcc6d9de4fa`
src/app/router.tsx	4,697 B	71	`6be406724a8908a317573dda0930e0f6353b42e795f533af7f58f7498e69f4db`
src/app/providers/AuthProvider.tsx	2,420 B	80	`eb99f91db794278f75c0fa0f7248e5b92a501ca907da4535e689881bfab6b269`
src/utils/quickDial.ts	149 B	3	`820c8ca7e5748bd60560660b9e0c86900aa5e8132ccd80b635e44e0944a8b1c2`
src/utils/excelExport.ts	2,482 B	85	`8625da5c7a1b8bb06b70183de7cf6fb2d9a9e3932fe5daddbc230600120918bb`
src/utils/agentPdfExport.ts	10,146 B	271	`b46f602cba6fd736eeda317929decab6a7d9cf0df1ec2f7c382fd0a44e080b11`
src/utils/businessPdfExport.ts	12,323 B	346	`b1c4a47269f17435abfe552b50af1bde7f29cac796269be283f4cdae8bfba191`

Prepared and Verified By:

_____

Senior Software Documentation Engineer
Date: 31st October 2026

Authorized Signatory for Applicant:

_____

Authorized Officer / Signatory
For: ARB SoftTech